

upsert なのに INSERT
が 2 回実行されていた



データベースの upsert 操作、便利ですね

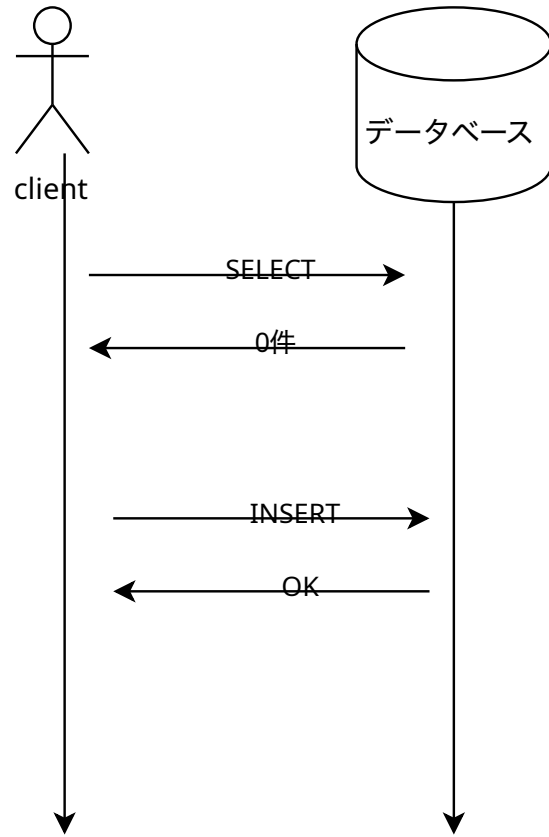
```
// 指定 email のユーザーがいれば更新、いなければ作成
await prisma.user.upsert({
  where: { email: "user@example.com" },
  update: { name: "admin" },
  create: { email: "user@example.com", name:
"admin" },
});
```

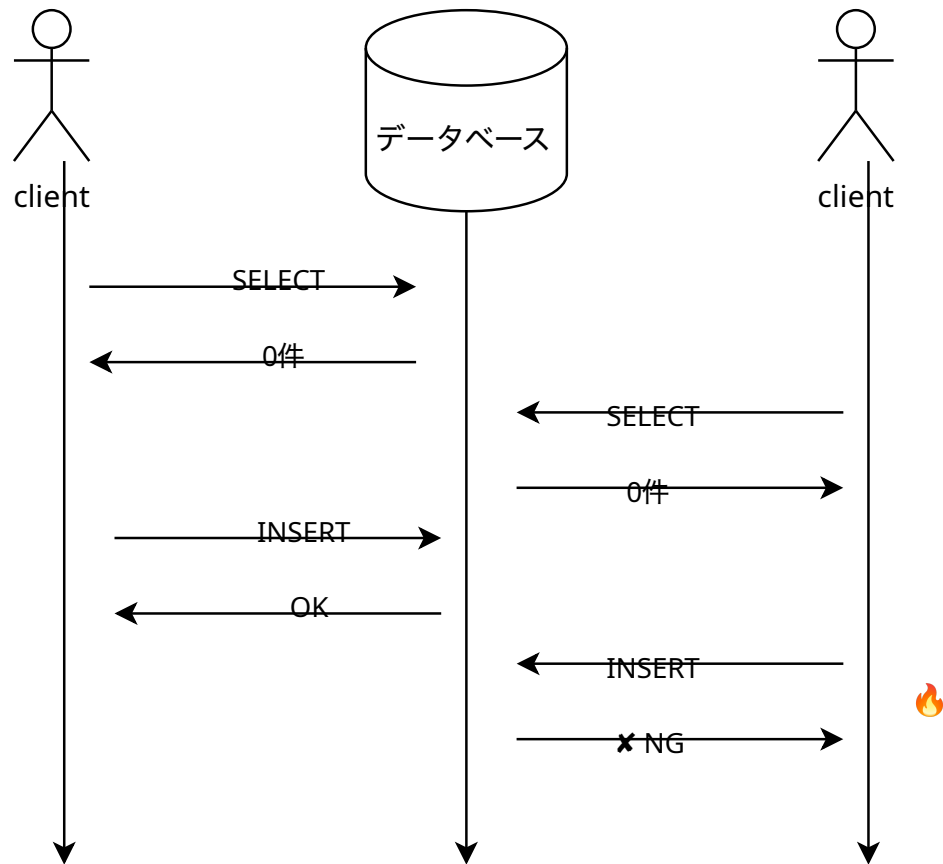
遭遇した事象

- たまに upsert 部分でユニーク制約エラーが出る
- すでにレコードがあるのに insert しようとする

??

- 発行される SQL を眺めると、SELECT + (INSERT or UPDATE) の 2 クエリになっている
- 同時に 2 つ SELECT を通過して、両方 INSERT している





ACTUALLY

2 クエリの upsert

-- 1. SELECT で存在確認

```
SELECT * FROM users WHERE email = 'user@example.com';
```

-- 2a. 見つかった → UPDATE

```
UPDATE users SET name = 'admin' WHERE email = 'user@example.com';
```

-- 2b. 見つからなかった → INSERT

```
INSERT INTO users (email, name) VALUES ('user@example.com', 'admin');
```

EXPECT

1 クエリの upsert

```
INSERT INTO users (email, name)
VALUES ('user@example.com', 'admin')
ON CONFLICT (email) DO UPDATE SET name = 'admin';
```

- なぜこうなる？ -- Prisma ORM の仕様

- データベース upsert は条件付きと書いてある

Database upserts

Where possible, Prisma Client hands over an `upsert` query to the database. This is called a *database upsert*.

Database upserts have the following advantages:

- They are faster than upserts handled by Prisma Client
- Unique key constraint errors cannot happen

Prisma Client uses a database upsert automatically when specific criteria are met. When these criteria are not met, Prisma Client handles the `upsert`.

To use a database upsert, Prisma Client sends the SQL construction `INSERT ... ON CONFLICT SET .. WHERE` to the database.

Database upsert prerequisites

Prisma Client uses database upserts with CockroachDB, PostgreSQL, or SQLite data sources.

<https://www.prisma.io/docs/orm/reference/prisma-client-reference#database-upserts>

- 要するに： 単一テーブルの操作の場合しか DB レベルの upsert にならない

- なぜこの仕様？ -- SQL の制限

column_name

The name of a column in the table named by ***table_name***. The column name can be qualified with a subfield name or array subscript, if needed. (Inserting into only some fields of a composite column leaves the other fields null.) When referencing a column with `ON CONFLICT DO UPDATE`, do not include the table's name in the specification of a target column. For example, `INSERT INTO table_name ... ON CONFLICT DO UPDATE SET table_name.col = 1` is invalid (this follows the general behavior for `UPDATE`).

<https://www.postgresql.org/docs/current/sql-insert.html>

- 要するに : DO UPDATE SET ... では同テーブルの
カラムしか操作できない

まとめ

- ORM はケースによって異なる SQL を発行することがある
- ドキュメントの記載が厚いところはちゃんと読もう

完